ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ

BÁO CÁO TIẾN ĐỘ BÀI TẬP LỚN

MÔN: ĐIỆN TOÁN ĐÁM MÂY

Mã lớp: INT3319 2

ĐỀ TÀI

SPOTPOOL – NỀN TẢNG CHO THUÊ TÀI NGUYÊN TÍNH TOÁN THEO JOB, ĐA NGƯỜI DÙNG, TÍNH PHÍ THEO MỨC SỬ DỤNG TRÊN AWS

Giảng viên hướng dẫn: TS. Hoàng Xuân Tùng

Nhóm thực hiện: G1

STT	Họ và tên	MSSV
1	Phạm Minh Nghĩa (Nhóm trưởng)	24021584
2	Nguyễn Minh Khuê	24021363
3	Đoàn Ngọc Anh	24021536

Hà Nội, 10/2026

MỤC LỤC

Chương 1. Tổng quan về dự án 2

1.1. Giới thiệu chung về tính năng 2

1.2. Người dùng của dự án 3

1.3. Quy trình sử dụng sản phẩm 4

1.4. Ví dụ sử dụng 6

Chương 2. Giải pháp thực hiện (dự kiến) 7

2.1. Kiến trúc tính năng 7

2.2. Kiến trúc triển khai 8

Chương 3. Kế hoạch triển khai và ước tính nỗ lực 13

3.1. Bảng hạng mục công việc 13

3.2. Tiến độ theo tuần 16

Tài liệu tham khảo 17

CHƯƠNG 1. TỔNG QUAN VỀ DỰ ÁN

1.1. Giới thiệu chung về tính năng

SpotPool là một nền tảng cho thuê sức tính toán theo job, xây dựng trên AWS theo mô hình PaaS. Người dùng không thuê nguyên một máy chủ. Họ nộp một công việc cần chạy (job), ví dụ nén một loạt video hay chạy một script xử lý dữ liệu, chọn gói dịch vụ, rồi nhận kết quả và hóa đơn. Hệ thống tự chọn máy, tự bật thêm hoặc tắt bớt máy theo lượng việc, và chỉ tính tiền theo thời gian job chạy thật. Người dùng không cần tài khoản AWS, không cần thẻ quốc tế và không cần hiểu các khái niệm như IAM, VPC hay Auto Scaling.

Nhóm không chỉ *sử dụng* cloud mà *xây dựng* một nền tảng cloud thu nhỏ ở mức PaaS trên hạ tầng AWS, tự hiện thực đủ năm đặc trưng theo định nghĩa của NIST: tự phục vụ theo nhu cầu (người dùng tự nộp job, tự nạp credit, không cần quản trị viên duyệt), truy cập rộng rãi qua mạng (cổng web, API và dòng lệnh), gộp tài nguyên (nhiều người dùng chung một cụm máy), co giãn nhanh (cụm máy tự tăng giảm theo lượng job, gói Tiết kiệm về 0 máy khi hết việc) và dịch vụ đo lường được (đo vCPU-giây và lập hóa đơn cho từng người). Để giảm chi phí, gói Tiết kiệm chạy trên máy Spot của AWS. Khi máy Spot bị thu hồi, job được chạy tiếp trên máy khác: job nén video chỉ phải làm lại video đang dở, còn job script chạy lại từ đầu. Job bị thu hồi quá 2 lần sẽ được chuyển sang máy On-Demand. Chi phí của SpotPool và của EC2 On-Demand tự thuê được đo trên hóa đơn AWS thật; so sánh với Fargate và AWS Batch được tính từ bảng giá.

Nền tảng cung cấp hai gói dịch vụ, khác nhau ở loại máy và đơn giá (Bảng 1.1).

Bảng 1.1. Hai gói dịch vụ của SpotPool

Đặc điểm	Ổn định	Tiết kiệm
Loại máy	On-Demand, luôn giữ sẵn 1 máy trong giờ hoạt động	Spot, hết việc thì về 0 máy
Mục tiêu thời gian chờ đến khi job bắt đầu chạy (SLA, áp dụng cho 95% số job)	≤ 1 phút	≤ 15 phút
Khi máy bị thu hồi	Không áp dụng	Job video làm lại video đang dở, job script chạy lại từ đầu; quá 2 lần thì chuyển sang On-Demand
Đơn giá	Cao	Thấp

Mọi gói dùng chung một đơn vị tài nguyên gọi là **suất**: 1 vCPU và 1,5 GB RAM. Mỗi máy 2 vCPU chạy cùng lúc 2 suất, có thể của hai người dùng khác nhau; job nặng có thể xin suất đôi (2 vCPU, 3 GB). Tiền được tính theo **vCPU-giây được cấp** (số vCPU của suất × số giây chạy), không theo mức CPU mà job dùng thật.

Các mục tiêu SLA ở trên là cam kết thiết kế; nhóm sẽ đo thời gian chờ thật ở tuần 4 và công bố cùng hóa đơn AWS. Đơn giá mỗi suất sẽ được tính từ chi phí máy thật trên

hóa đơn AWS, chia theo hệ số sử dụng của cụm, cộng phần dự phòng cho job chạy lại và biên lợi nhuận; số cụ thể được chốt sau khi đo ở tuần 3–4. Tiền trên nền tảng là credit ảo cấp cho người dùng; thanh toán thật nằm ngoài phạm vi môn học.

So với AWS Batch, dịch vụ chạy job có sẵn của AWS, SpotPool có thêm lớp nhà cung cấp mà AWS Batch không có: người dùng không cần tài khoản AWS, chỉ cần đăng ký bằng email, chọn gói, đặt trần chi phí và nhận hóa đơn riêng tính theo vCPU-giây. Phần chạy job bên dưới (hàng đợi, tự co giãn, xử lý máy Spot bị thu hồi) nhóm tự xây thay vì dùng AWS Batch để hiểu và đo được từng cơ chế.

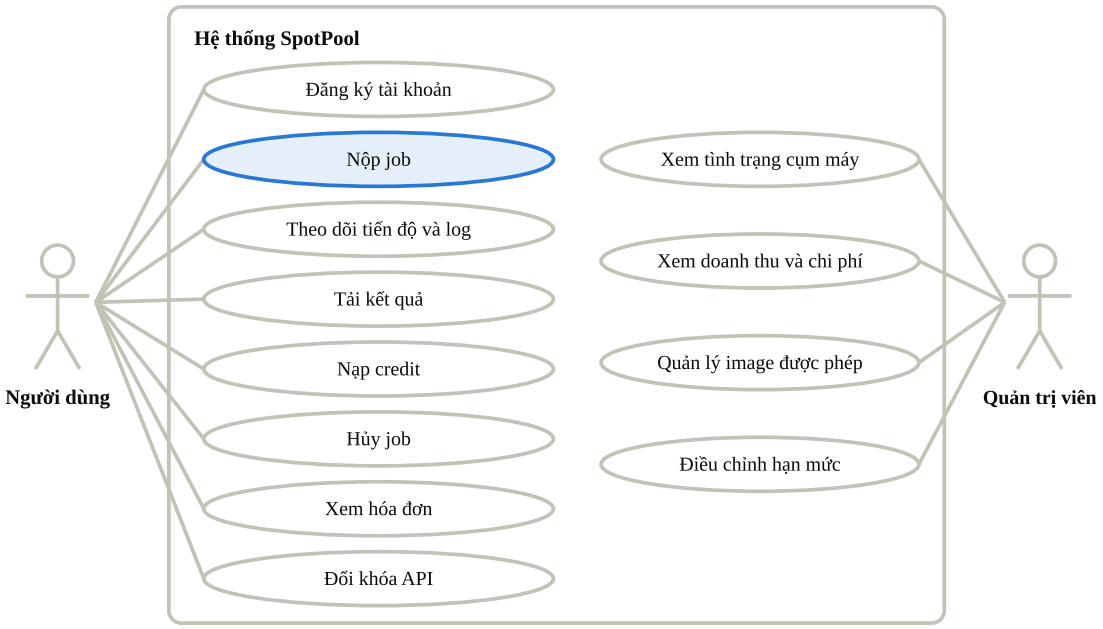
1.2. Người dùng của dự án

SpotPool hướng tới các nhóm người dùng có nhu cầu thuê sức tính toán linh hoạt. Bảng 1.2 mô tả chi tiết ba nhóm đối tượng chính.

Bảng 1.2. Các nhóm người dùng của SpotPool

Nhóm người dùng	Ví dụ	Nhu cầu	Khó khăn khi tự dùng AWS
Sinh viên, học viên	Làm đồ án xử lý ảnh, video, dữ liệu	Thỉnh thoảng chạy job dài hoặc nhiều việc song song mà không phải treo máy cá nhân	Không có thẻ quốc tế, sợ hóa đơn bất ngờ, quota tài khoản mới thấp
Nhóm nghiên cứu nhỏ	Chạy mô phỏng tính toán; huấn luyện mô hình học máy truyền thống; tìm siêu tham số song song	Chạy nhiều job theo đợt, chi phí thấp, chấp nhận chờ	Phải tự dựng và quản lý cụm máy, khó xử lý Spot bị thu hồi
Nhà phát triển, startup nhỏ	Chuyển đổi video cho ứng dụng, xử lý báo cáo định kỳ	Gọi qua API, biết trước chi phí tối đa	Không muốn tốn công vận hành hạ tầng, máy hay ngồi không

Nền tảng chỉ dùng máy CPU nên không nhắm tới huấn luyện học sâu cần GPU. Người rành AWS có thể tự dùng AWS Batch với giá gốc tương đương. SpotPool nhắm đến người dùng nhỏ cần sự đơn giản, tương tự cách Heroku hay Vercel được xây dựng trên AWS.



Hình 1.1. Biểu đồ use case của SpotPool

Người dùng làm mọi việc qua web hoặc API mà không cần quản trị viên duyệt. Khi AWS thu hồi máy Spot, hệ thống tự xử lý để job chạy tiếp mà không ai phải thao tác, nên bước này không có trong biểu đồ mà được thể hiện ở Hình 1.2.

1.3. Quy trình sử dụng sản phẩm

Người dùng đạt mục đích "chạy được job với chi phí biết trước" qua năm quy trình dưới đây. Hình 1.2 ở cuối mục minh họa vòng đời của một job.

Quy trình 1 – Đăng ký và nhận credit

1. Người dùng đăng ký bằng email trên cổng web. Hệ thống sinh một khóa API và chỉ hiện một lần để người dùng lưu lại; khóa này dùng để đăng nhập cổng web.
2. Hệ thống tạo hồ sơ, cấp số dư credit ban đầu và hạn mức mặc định (số job chạy đồng thời, số job lỗi miễn phí mỗi ngày).
3. Người dùng dùng chính khóa API đó để gọi từ dòng lệnh hoặc từ ứng dụng của họ, và có thể đổi khóa mới khi lộ khóa cũ.

Quy trình 2 – Nộp job

1. Chọn loại job: nén video (mẫu có sẵn) hoặc chạy script của người dùng trong một image thuộc danh sách được phép.
2. Tải dữ liệu đầu vào lên. File đi thẳng vào kho lưu trữ qua đường dẫn tải lên có thời hạn, không qua máy chủ trung gian.
3. Chọn gói Ổn định hoặc Tiết kiệm. Với job script chạy lâu, hệ thống nhắc rằng ở gói Tiết kiệm, nếu máy bị thu hồi thì script phải chạy lại từ đầu.
4. Đặt **trần chi phí** (số credit tối đa). Hệ thống quy đổi ra thời gian chạy tối đa theo đơn giá gói. Với mẫu nén video, hệ thống gợi ý mức chi phí dựa trên lịch

sử chạy; với script tự viết, người dùng có thể chạy thử trên một phần nhỏ dữ liệu trước.

5. Bấm Chạy. Hệ thống kiểm tra số dư và hạn mức rồi đưa job vào hàng đợi của gói đã chọn.

Quy trình 3 – Theo dõi và nhận kết quả

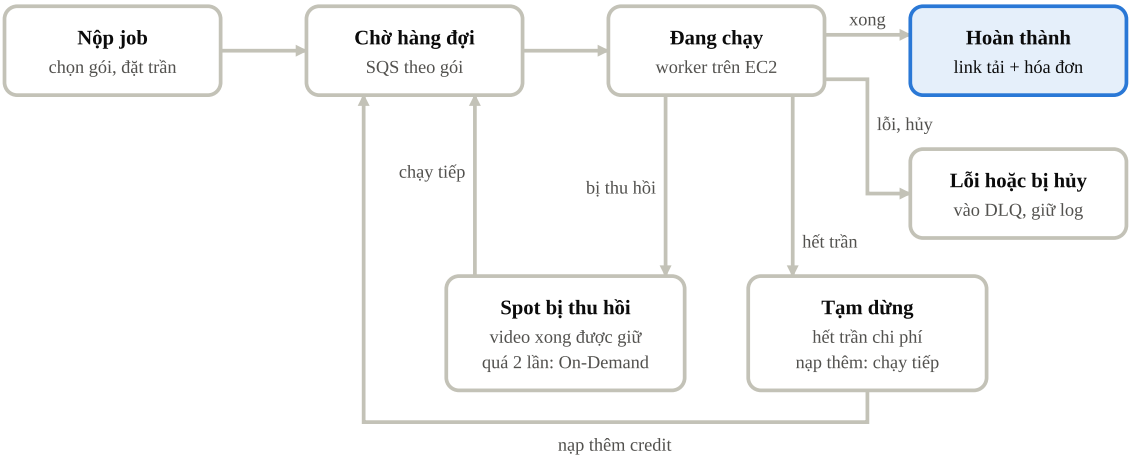
1. Trang chi tiết job hiển thị trạng thái theo thời gian thực: đang chờ, đang chạy, bị gián đoạn và chạy tiếp, hoàn thành.
2. Người dùng xem log và tiến độ (với job video: số video đã xong).
3. Khi hoàn thành, kết quả được giữ 7 ngày để người dùng tải về lúc nào cũng được (mỗi lần bấm tải, hệ thống cấp một link tạm thời), kèm chi phí thực tế của job.

Quy trình 4 – Xử lý khi có sự cố

- *Máy Spot bị thu hồi:* hệ thống tự xử lý, người dùng chỉ thấy trạng thái "bị gián đoạn, đang chạy tiếp". Phần chạy lại do nền tảng chịu, không tính vào hóa đơn.
- *Hết trần chi phí:* job tạm dừng. Với job video, các video đã xong vẫn được giữ, người dùng nạp thêm để chạy nốt phần còn lại; với job script, người dùng chỉ trả phần đã chạy.
- *Job lỗi hoặc bị hủy:* miễn phí trong hạn mức số lần mỗi ngày. Người dùng vẫn tải được log và kết quả dở dang.

Quy trình 5 – Xem hóa đơn và quản trị

1. Người dùng xem hóa đơn theo từng job: gói, số vCPU-giây, đơn giá, thành tiền, cùng biểu đồ mức dùng theo ngày.
2. Quản trị viên xem trang tổng quan: số máy đang chạy theo loại (Spot, On-Demand) và vùng sẵn sàng (AZ), số job chờ trong từng hàng đợi, doanh thu từ người dùng so với chi phí trả cho AWS.



Hình 1.2. Vòng đời của một job

Job bị thu hồi quay về hàng đợi, các video đã xong được giữ lại; job hết trần tạm dừng chờ nạp thêm. Chỉ job lỗi hoặc bị hủy mới rời luồng chính.

1.4. Ví dụ sử dụng

Ví dụ sau dùng số liệu giả định để minh họa cả quy trình; đơn giá thật sẽ được đặt sau khi đo chi phí.

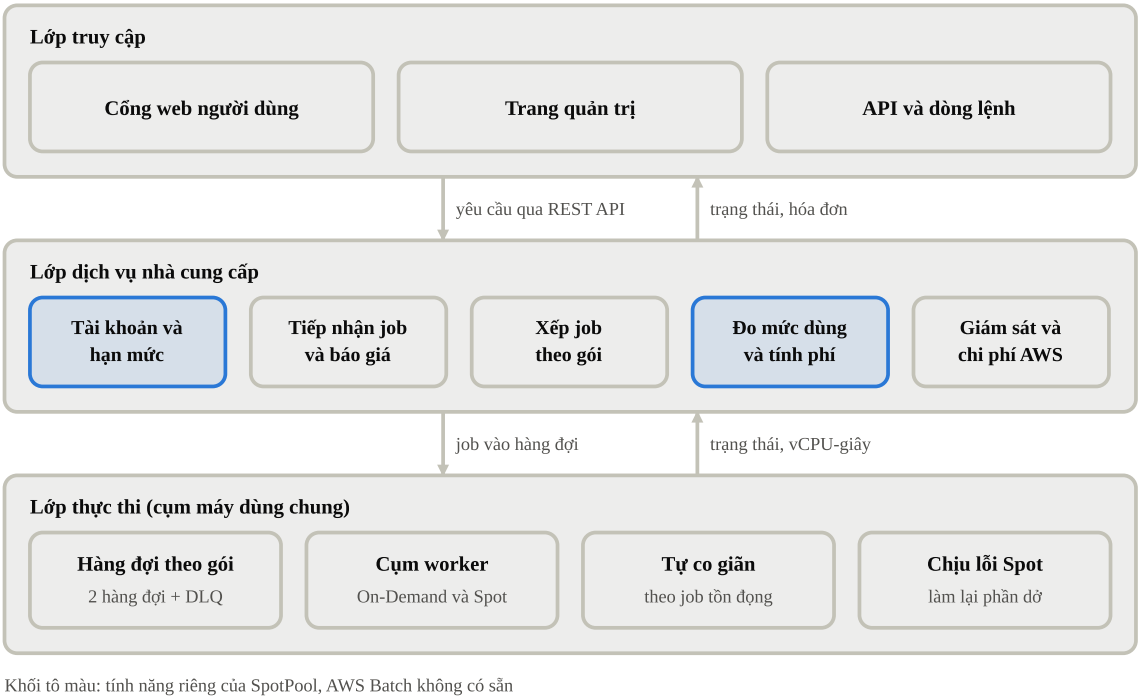
1. **Nhu cầu.** Lan, sinh viên năm 3, cần nén 20 video bài giảng (mỗi video 10 phút) xuống 720p để nộp lên hệ thống học tập của trường. Việc không gấp, chỉ cần xong trước 8 giờ sáng hôm sau.
2. **Nộp job.** Lan chọn mẫu "Nén video", tải 20 file lên, chọn gói Tiết kiệm vì việc không gấp. Dựa trên lịch sử chạy, hệ thống ước lượng job cần khoảng 200 suất-phút, tức khoảng 1 suất-phút cho mỗi phút video. Với đơn giá giả định 50 đ/suất-phút, chi phí dự kiến khoảng 10.000 đ; Lan đặt trần 15.000 đ.
3. **Hệ thống xử lý.** Mỗi video là một việc riêng, 20 việc được đưa vào hàng đợi Tiết kiệm. Cụm Tiết kiệm đang ở 0 máy nên tự bật 4 máy Spot, tức 8 suất, nén 8 video cùng lúc.
4. **Sự cố.** Giữa chừng, AWS thu hồi một máy. Hai video đang nén dở trên máy đó quay lại hàng đợi và được máy khác nén lại từ đầu; các video đã xong không bị ảnh hưởng. Lan chỉ thấy trạng thái "bị gián đoạn, đang chạy tiếp".
5. **Kết quả.** Khoảng 30 phút sau, cả 20 video xong. Lan nhận link tải và hóa đơn khoảng 10.000 đ; phần chạy lại do máy bị thu hồi không bị tính tiền.
6. **Giải phóng tài nguyên.** Hết việc, cụm Tiết kiệm tự giảm về 0 máy, nền tảng không tốn thêm tiền máy.

Ví dụ này đi qua đủ năm đặc trưng cloud của NIST: Lan tự phục vụ qua mạng, các máy được dùng chung với người dùng khác, cụm máy co giãn từ 0 lên 4 rồi về 0, và mức dùng được đo để tính đúng phần đã chạy.

CHƯƠNG 2. GIẢI PHÁP THỰC HIỆN (DỰ KIẾN)

2.1. Kiến trúc tính năng

Hệ thống chia thành ba lớp: lớp truy cập (người dùng tương tác), lớp dịch vụ nhà cung cấp (quản lý khách hàng, xếp job, tính phí) và lớp thực thi (chạy job trên cụm máy dùng chung). Hình 2.1 thể hiện các khối chức năng và hướng dữ liệu giữa chúng.



Hình 2.1. Kiến trúc tính năng theo ba lớp

Yêu cầu đi từ trên xuống qua API và hàng đợi; trạng thái và mức dùng đi ngược lên để tính phí và hiển thị.

Lớp truy cập. Cổng web cho người dùng (đăng ký, nộp job, theo dõi, hóa đơn), trang quản trị cho nhóm vận hành, và API/dòng lệnh cho người dùng kỹ thuật. Cả ba dùng chung một bộ API nên mọi thao tác trên web đều làm được bằng API.

Lớp dịch vụ nhà cung cấp gồm năm khối:

- *Tài khoản và hạn mức:* xác thực người dùng, lưu số dư credit, giới hạn số job đồng thời và số job lỗi miễn phí.
- *Tiếp nhận job và báo giá:* kiểm tra image có trong danh sách cho phép, quy đổi trần chi phí ra thời gian chạy tối đa, tạm giữ credit, cấp đường dẫn tải dữ liệu lên.
- *Xếp job theo gói:* đưa job vào hàng đợi đúng gói; job video được tách thành nhiều việc, mỗi video một việc. Job bị thu hồi quá 2 lần được chuyển sang hàng đợi On-Demand. Job bị kẹt khi máy sập nguồn hay đứt mạng được xử lý mà không cần bộ lập lịch riêng: worker cập nhật nhịp tim (heartbeat) lên

DynamoDB mỗi 30 giây; khi worker chết, thông điệp của job tự hiện lại trên hàng đợi sau visibility timeout, và worker khác được giành lại job nếu đã quá 2 phút không có nhịp tim. Lượt chạy hỏng không tính tiền.

- *Đo mức dùng và tính phí:* cộng vCPU-giây được cấp cho các phần chạy thành công, trừ credit, áp dụng chính sách miễn phí job lỗi, tạm dừng job khi hết trần. Khối này cũng ghi lại "chi phí rủi ro" nền tảng phải gánh.
- *Giám sát và báo cáo chi phí:* thu số liệu hàng đợi, số máy, số lần thu hồi và chi phí AWS để hiển thị trên dashboard và phục vụ bài toán hòa vốn.

Lớp thực thi gồm bốn khối:

- *Hàng đợi theo gói:* hai hàng đợi Ổn định và Tiết kiệm, cùng một hàng đợi job lỗi (DLQ). Các worker cùng kéo việc từ hàng đợi (competing consumers), nên tải được cân bằng tự nhiên mà không cần bộ cân bằng tải. Vì SQS Standard có thể giao một thông điệp nhiều lần, worker phải giành job trước khi chạy bằng một lệnh cập nhật có điều kiện trên DynamoDB, chỉ thành công khi job đang chờ hoặc đã mất nhịp tim quá 2 phút. Worker giành hụt thì bỏ thông điệp đó.
- *Cụm worker:* mỗi gói một nhóm máy 2 vCPU, mỗi máy chạy cùng lúc 2 suất. Trên mỗi máy, một tiến trình điều phối (worker agent) lấy job, chạy job trong container bị giới hạn đúng bằng suất được cấp (1 vCPU, 1,5 GB RAM), giới hạn thời gian và không có mạng ra ngoài, sau đó ghi kết quả và mức dùng. Vì job của hai người dùng có thể chạy chung một máy, các giới hạn này giữ cho job này không làm chậm và không đọc được dữ liệu của job kia.
- *Tự co giãn:* chia số job đang chờ cho số máy đang chạy (backlog-per-instance, cách AWS khuyến nghị khi co giãn theo hàng đợi SQS) để tăng giảm số máy; có luật riêng để bật máy đầu tiên khi nhóm đang ở 0 máy; nhóm Ổn định giữ sẵn 1 máy trong giờ hoạt động.
- *Chịu lỗi khi Spot bị thu hồi:* worker hỏi cảnh báo thu hồi của AWS mỗi 5 giây (AWS báo trước khoảng 2 phút). Khi nhận cảnh báo, worker ngừng nhận việc mới và trả việc đang dở về hàng đợi ngay. Job nén video được chia theo từng video (task chunking): mỗi video là một việc độc lập, nên máy bị thu hồi thì chỉ phải làm lại video đang dở. Job script chạy lại từ đầu; bị thu hồi quá 2 lần thì được chuyển sang máy On-Demand. Theo AWS Spot Instance Advisor, tần suất thu hồi Spot trung bình trong lịch sử dưới 5% (tính theo 30 ngày gần nhất); nhóm vẫn thiết kế chịu lỗi vì tỷ lệ này thay đổi theo thời điểm, và dùng AWS FIS để chủ động giả lập thu hồi khi kiểm thử vì trong 5 tuần rất có thể không gặp lần thu hồi thật nào.

2.2. Kiến trúc triển khai

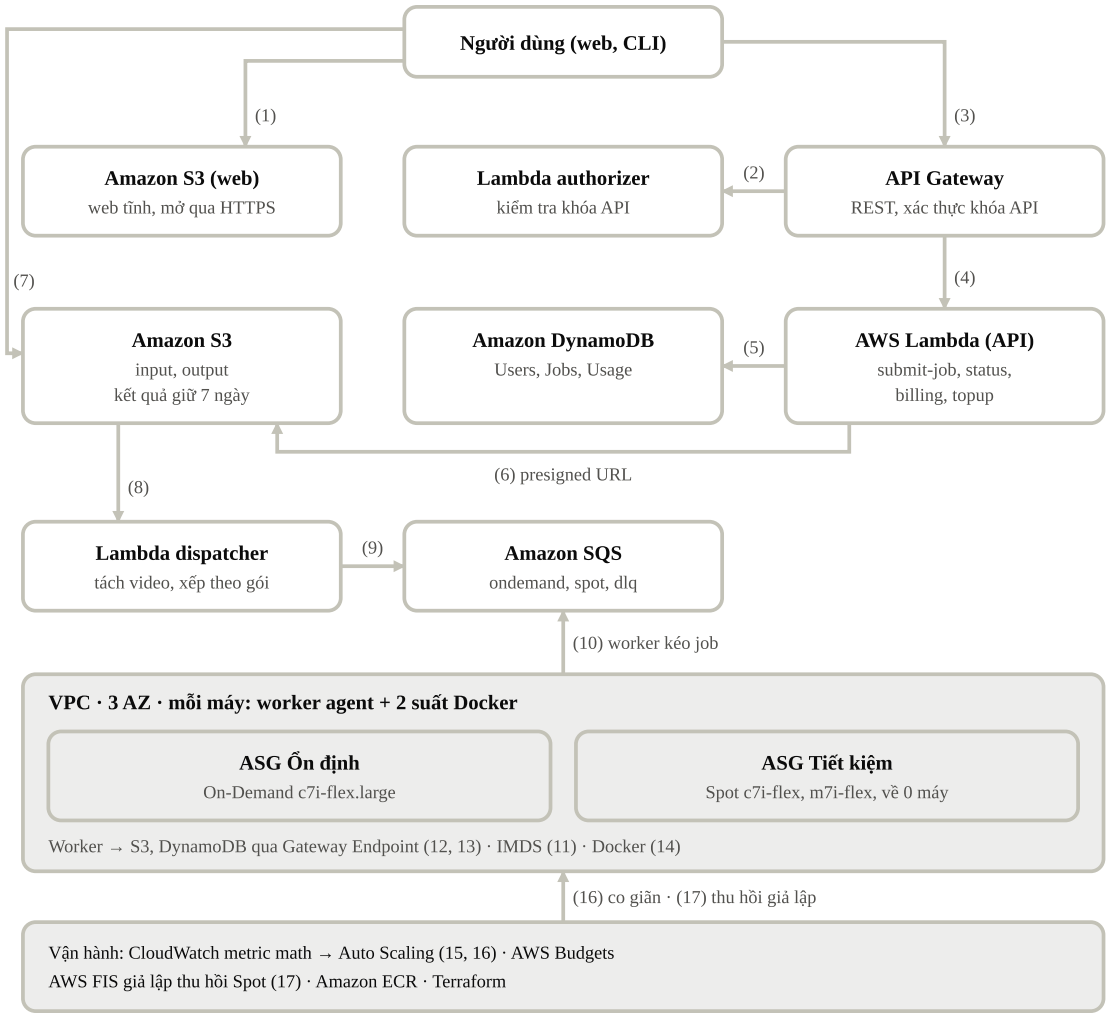
Toàn bộ hệ thống chạy trong một tài khoản AWS, một region (dự kiến ap-southeast-1 ở Singapore, gần Việt Nam; chốt sau khi so giá Spot ở tuần 1), trải trên cả 3 vùng sẵn sàng (AZ) của region, và được dựng hoàn toàn bằng Terraform. Lớp truy cập và lớp

dịch vụ dùng dịch vụ serverless (không tốn máy chạy liên tục); chỉ lớp thực thi dùng máy EC2. Hình 2.2 thể hiện các thành phần triển khai.

Bảng 2.1 ánh xạ từng khối chức năng sang dịch vụ AWS triển khai.

Bảng 2.1. Ánh xạ khối chức năng sang dịch vụ AWS

Khối chức năng	Dịch vụ AWS triển khai
Cổng web, trang quản trị	Ứng dụng web tĩnh (React) đặt trên Amazon S3, mở qua đường dẫn HTTPS của S3
API, dòng lệnh	Amazon API Gateway (REST) + AWS Lambda
Tài khoản và hạn mức	Lambda authorizer của API Gateway, so khóa API với mã băm lưu trong bảng `Users` trên Amazon DynamoDB
Tiếp nhận job, báo giá	Lambda `submit-job`, bảng `Jobs` trên DynamoDB, S3 bucket `input`
Xếp job theo gói	Lambda `dispatcher` (kích hoạt bởi S3 Event)
Hàng đợi theo gói	Amazon SQS: `sqs-ondemand`, `sqs-spot`, `sqs-dlq`
Cụm worker	2 Auto Scaling Group gồm máy EC2 2 vCPU chạy worker agent và Docker; image lưu trên Amazon ECR. Gói Ổn định dùng On-Demand `c7i-flex.large`; gói Tiết kiệm dùng Spot với chiến lược price-capacity-optimized. Tài khoản đang ở gói Free của AWS (200 USD credit, chi phí trừ dần vào credit) nên chỉ dùng được một số loại máy như `c7i-flex.large`, `m7i-flex.large`; nếu nâng lên gói Paid, nhóm mở rộng ra khoảng 10 loại máy x86 2 vCPU theo khuyến nghị của AWS. Khi đo so sánh chi phí, nhóm cố định `c7i-flex.large` để kết quả công bằng
Tự co giãn	Target tracking trên chỉ số backlog-per-instance tính bằng CloudWatch metric math (số job chờ ÷ số máy đang chạy), không cần hàm tự đẩy chỉ số; một CloudWatch alarm bật máy đầu tiên khi nhóm đang ở 0; scheduled scaling giữ 1 máy Ổn định trong giờ hoạt động
Kết quả	S3 bucket `output`, lifecycle rule tự xóa sau 7 ngày
Đo mức dùng, tính phí	Bảng `Usage` trên DynamoDB, Lambda `billing`
Mạng	VPC trải 3 AZ, public subnet, S3 Gateway Endpoint và DynamoDB Gateway Endpoint
Giám sát, chi phí, kiểm thử lỗi	CloudWatch dashboard và log, AWS Budgets, Cost Explorer với cost allocation tag, AWS FIS



Hình 2.2. Kiến trúc triển khai trên AWS

Dữ liệu lớn đi thẳng giữa người dùng, worker và S3; Lambda chỉ trao đổi thông tin điều khiển nhỏ qua DynamoDB và SQS.

Mạng và chi phí. Worker đặt trong public subnet của cả 3 AZ, chặn toàn bộ kết nối đi vào bằng security group. Worker đọc ghi S3 và DynamoDB qua Gateway Endpoint (miễn phí) và không dùng NAT Gateway. Mỗi máy có một địa chỉ IPv4 công khai để gọi SQS, ECR và CloudWatch; từ tháng 2/2024, AWS tính 0,005 USD/giờ cho mỗi địa chỉ này ([6]). Nhóm chấp nhận khoản này vì rẻ hơn NAT Gateway hay VPC Interface Endpoint khi cụm chỉ có vài máy. Truyền dữ liệu giữa EC2 và S3 trong cùng region không tính phí, nên rải máy Spot ra nhiều AZ để tăng khả năng có máy mà không tốn thêm phí truyền.

Cách ly người dùng. Chỉ image trong ECR thuộc danh sách cho phép mới được chạy. Mỗi container bị giới hạn CPU, RAM, thời gian chạy và chạy với `--network none`; dữ liệu vào ra được worker agent chuẩn bị sẵn trong thư mục riêng. Worker agent chỉ được cấp quyền IAM tối thiểu cần cho hàng đợi, bucket và bảng của mình.

Khởi động nhanh và giữ SLA. Máy được bật từ Custom AMI do nhóm tạo một lần từ một máy mẫu, đã có Docker, worker agent và image mẫu, nên bỏ được bước cài đặt khi khởi động. Dù vậy, bật một máy mới vẫn mất từ một đến vài phút, gồm thời gian Auto Scaling phát hiện tải, khởi tạo và boot máy. Vì thế mục tiêu ≤ 1 phút của gói Ổn định được giữ bằng 1 máy On-Demand chạy sẵn trong giờ hoạt động.

Bảng 2.2 liệt kê giao diện tương tác giữa các thành phần; số thứ tự trùng với số trong ngoặc ở Hình 2.2.

Bảng 2.2. Giao diện tương tác giữa các thành phần

STT	Từ → Đến	Giao diện, giao thức	Nội dung trao đổi
1	Trình duyệt → S3	HTTPS	Tải ứng dụng web tĩnh
2	API Gateway → Lambda authorizer	Gọi trước khi chuyển yêu cầu, có lưu đệm kết quả	Khóa API trong header; trả về userId nếu khóa hợp lệ
3	Web, CLI → API Gateway	REST/JSON qua HTTPS, header `Authorization: Bearer <khóa API>`	`POST /jobs`, `GET /jobs/{id}`, `POST /jobs/{id}/cancel`, `POST /jobs/{id}/topup`, `GET /billing`
4	API Gateway → Lambda	Lambda proxy integration	Yêu cầu đã xác thực, kèm `userId` do authorizer trả về
5	Lambda → DynamoDB	AWS SDK: PutItem, Query, UpdateItem có điều kiện	Tạo job, tạm giữ credit, kiểm tra hạn mức một cách nguyên tử
6	Lambda → S3	AWS SDK, tạo presigned URL	Link tải đầu vào (PUT) và link tải kết quả (GET) có thời hạn
7	Người dùng → S3	HTTPS PUT/GET theo presigned URL	Dữ liệu đầu vào, kết quả
8	S3 `input` → Lambda `dispatcher`	S3 Event Notification (ObjectCreated)	Báo dữ liệu đã lên đủ, bắt đầu xếp hàng job
9	Lambda → SQS	SendMessage	Thông điệp việc: `jobId`, loại job (video, script), gói, file đầu vào, image, tham số, thời gian tối đa, số lần bị thu hồi
10	Worker → SQS	ReceiveMessage (long polling), ChangeMessageVisibility, SendMessage, DeleteMessage	Nhận việc; gia hạn thời gian giữ việc khi chạy lâu; đặt về 0 để trả việc khi bị thu hồi; chuyển sang `sqs-ondemand` khi bị thu hồi quá 2 lần; xóa khi xong
11	Worker → Instance Metadata (IMDSv2)	HTTP nội bộ, hỏi mỗi 5 giây	Phát hiện cảnh báo thu hồi Spot (`spot/instance-action`)
12	Worker → S3 qua Gateway Endpoint	AWS SDK	Tải đầu vào; ghi kết quả
13	Worker → DynamoDB qua Gateway Endpoint	AWS SDK: UpdateItem có điều kiện	Giành job (đang chờ, hoặc đã mất nhịp tim quá 2 phút); trạng thái, tiến độ, nhịp tim, bản ghi vCPU-giây
14	Worker agent → Docker	Docker Engine API qua Unix socket	Chạy container theo suất: `--cpus=1`, `--memory=1.5g`, `--network none`, giới hạn thời gian

STT	Từ → Đến	Giao diện, giao thức	Nội dung trao đổi
15	SQS, Auto Scaling → CloudWatch	Chỉ số có sẵn, CloudWatch metric math	Số job chờ (`ApproximateNumberOfMessagesVisible`) chia số máy đang chạy (`GroupInServiceInstances`) ra backlog-per-instance
16	CloudWatch → Auto Scaling	Target tracking, CloudWatch alarm	Tăng giảm số máy mỗi nhóm; bật máy đầu tiên khi đang ở 0
17	AWS FIS → EC2 Spot	Thí nghiệm `aws:ec2:send-spot-instance-interruptions`	Giả lập thu hồi máy Spot khi kiểm thử và demo

CHƯƠNG 3. KẾ HOẠCH TRIỂN KHAI VÀ ƯỚC TÍNH NỖ LỰC

Đến ngày 11/10/2026, nhóm đã chốt đề tài [Minh chứng], hoàn thành thiết kế dự kiến (Chương 1–2) và xong hạng mục 1 (kiểm tra tài khoản AWS chính). Kết quả: quota 32 vCPU cho Spot và 16 vCPU cho On-Demand; gói Free không cho dùng c7i.large nên nhóm chuyển sang c7i-flex.large (cùng 2 vCPU, 4 GB); máy c7i-flex.large chạy được cả dạng Spot lẫn On-Demand, và AWS ước tính bản Spot rẻ hơn On-Demand khoảng 63%. Nhóm có 5 tuần để hoàn thành; lưu trạng thái (checkpoint) và lập lịch theo hạn chót để lại làm hướng mở rộng. Kế hoạch gồm 21 hạng mục, tổng **165 giờ-người**; cộng 15% dự phòng thành **190 giờ-người**, tức khoảng 13 giờ mỗi người mỗi tuần. Chia theo người (đã gồm dự phòng): TV1 (nhóm trưởng) 68 giờ, TV2 59 giờ, TV3 63 giờ. TV1 nhận thêm phần điều phối nhóm (hạng mục 21); TV2 ít giờ nhất nhưng giữ worker agent, phần khó nhất.

Nỗ lực được ước lượng theo từng hạng mục nhỏ (WBS), dựa trên số thành phần và giao diện mỗi hạng mục phải làm. Khâu ghép nối và sửa lỗi được tách riêng 12 giờ vì đây là khâu dễ thiếu giờ nhất. Các hạng mục chủ yếu là cấu hình hoặc giao diện (3, 9, 10, 13, 15, 16) được ước lượng thấp hơn vì nhóm dùng các tài nguyên có sẵn. Cột "Thành phần kiến trúc" chỉ rõ hạng mục thuộc khối nào trong Hình 2.1 và dịch vụ nào trong Hình 2.2; TV1, TV2, TV3 là ba thành viên theo thứ tự ở trang bìa.

3.1. Bảng hạng mục công việc

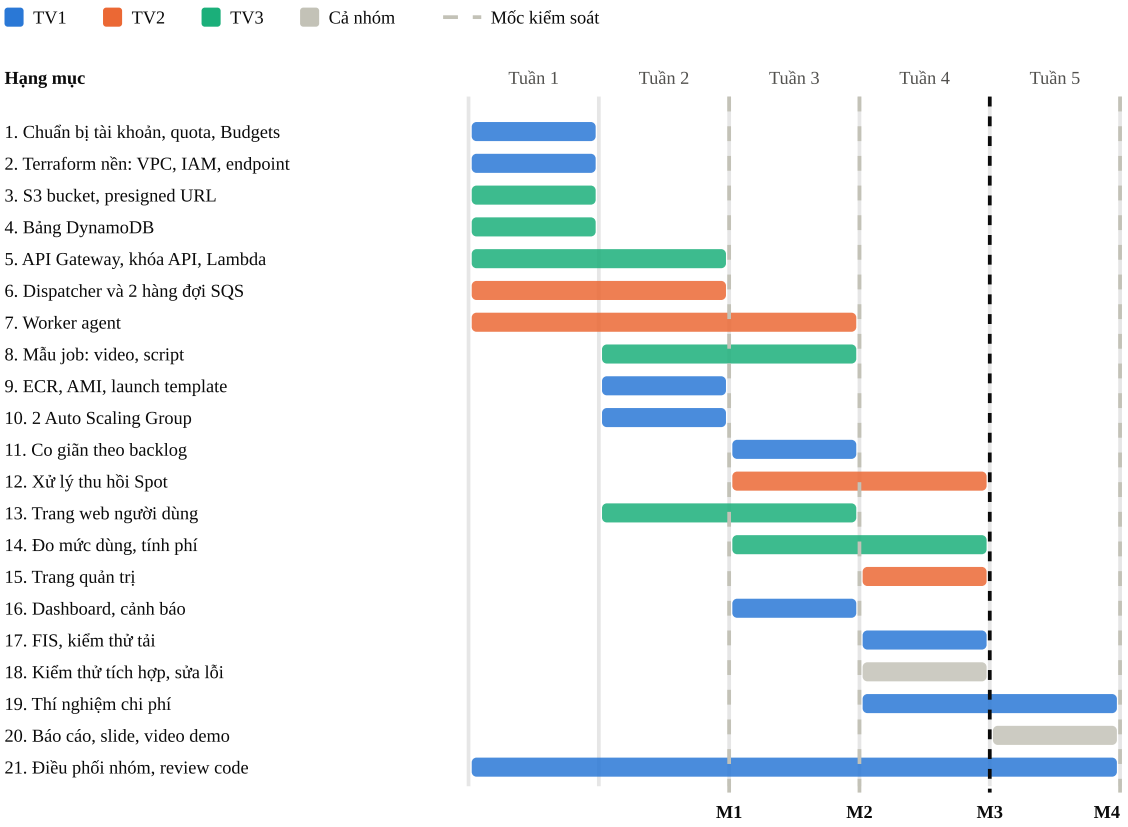
Bảng 3.1. Các hạng mục công việc và ước tính nỗ lực

STT	Hạng mục công việc	Thành phần kiến trúc liên quan	Tuần	Nỗ lực (giờ-người)	Phụ trách	Trạng thái
1	Kiểm tra tài khoản: đặt AWS Budgets, xem quota Spot và On-Demand, bật thử máy Spot và On-Demand	Cụm worker (quota Spot, On-Demand); Giám sát và báo cáo chi phí (AWS Budgets)	1	1	TV1	Hoàn thành [Minh chứng]
2	Terraform nền: VPC 3 AZ, subnet, security group, S3 và DynamoDB Gateway Endpoint, IAM role, lưu state	Mạng và bảo mật (Hình 2.2)	1	10	TV1	Chưa bắt đầu
3	Tạo S3 bucket `input`, `output`, lifecycle rule xóa sau 7 ngày, hàm tạo presigned URL	Tiếp nhận job; Kết quả (Amazon S3)	1	4	TV3	Chưa bắt đầu
4	Thiết kế bảng DynamoDB `Users`, `Jobs`, `Usage`	Tài khoản và hạn mức; Đo mức dùng (DynamoDB)	1	5	TV3	Chưa bắt đầu

STT	Hạng mục công việc	Thành phần kiến trúc liên quan	Tuần	Nỗ lực (giờ-người)	Phụ trách	Trạng thái
5	API Gateway, Lambda authorizer kiểm tra khóa API, Lambda đăng ký và Lambda cho tạo job, xem trạng thái, hủy job (giao diện 2–6)	Lớp truy cập; Tài khoản và hạn mức	1–2	12	TV3	Chưa bắt đầu
6	Lambda `dispatcher` tách job video thành từng video, 2 hàng đợi SQS và DLQ (giao diện 8–9)	Xếp job theo gói; Hàng đợi theo gói	1–2	6	TV2	Chưa bắt đầu
7	Worker agent: long polling, giành job trên DynamoDB, gửi nhịp tim, chạy 2 suất song song trong container cách ly, gia hạn visibility, ghi kết quả và trạng thái (giao diện 10, 12–14)	Cụm worker (EC2, Docker)	1–3	22	TV2	Chưa bắt đầu
8	Hai mẫu job: nén video bằng FFmpeg (mỗi video một việc) và chạy script Python	Cụm worker; Phân mảnh tác vụ	2–3	6	TV3	Chưa bắt đầu
9	Amazon ECR, danh sách image được phép, Custom AMI tạo từ máy mẫu và launch template	Cụm worker; Cách ly người dùng	2	4	TV1	Chưa bắt đầu
10	2 Auto Scaling Group: On-Demand cho gói Ổn định; Spot nhiều AZ, price-capacity-optimized cho gói Tiết kiệm	Tự co giãn; Cụm worker	2	6	TV1	Chưa bắt đầu
11	Co giãn theo backlog-per-instance bằng metric math, luật bật từ 0 máy, giữ 1 máy Ổn định trong giờ hoạt động (giao diện 15–16)	Tự co giãn (CloudWatch, Auto Scaling)	3	8	TV1	Chưa bắt đầu
12	Xử lý thu hồi Spot: bắt cảnh báo 2 phút, trả việc đang dở về hàng đợi, đếm số lần thu hồi, quá 2 lần thì chuyển sang On-Demand (giao diện 10–11)	Chịu lỗi khi Spot bị thu hồi	3–4	10	TV2	Chưa bắt đầu
13	Trang web người dùng: đăng ký, đăng nhập bằng khóa API, tạo job, chi tiết job, hóa đơn	Lớp truy cập (S3)	2–3	10	TV3	Chưa bắt đầu
14	Đo vCPU-giây, trừ credit, trần chi phí, nạp thêm, chính sách miễn phí job lỗi	Đo mức dùng và tính phí (Lambda `billing`, bảng `Usage`)	3–4	10	TV3	Chưa bắt đầu
15	Trang quản trị: số máy, hàng đợi, doanh thu so với chi phí AWS	Lớp truy cập; Giám sát và báo cáo chi phí	4	5	TV2	Chưa bắt đầu

STT	Hạng mục công việc	Thành phần kiến trúc liên quan	Tuần	Nỗ lực (giờ-người)	Phụ trách	Trạng thái
16	CloudWatch dashboard, cost allocation tag, cảnh báo	Giám sát và báo cáo chi phí	3	3	TV1	Chưa bắt đầu
17	Kịch bản AWS FIS thu hồi Spot và kiểm thử tải (giao diện 17)	Chịu lỗi khi Spot bị thu hồi; Tự co giãn	4	6	TV1	Chưa bắt đầu
18	Kiểm thử tích hợp toàn hệ thống và sửa lỗi	Cả ba lớp (Hình 2.1); giao diện 1–17	4	12	Cả nhóm	Chưa bắt đầu
19	Thí nghiệm chi phí: đo thật SpotPool so với EC2 On-Demand tự thuê trên hóa đơn AWS; so với Fargate và AWS Batch bằng tính toán từ bảng giá; đo thời gian chờ; tính điểm hòa vốn	Giám sát và báo cáo chi phí; Đo mức dùng	4–5	8	TV1	Chưa bắt đầu
20	Báo cáo cuối kỳ, slide, quay video demo dự phòng	Toàn bộ kiến trúc (Hình 2.1, 2.2): trình bày và demo	5	12	Cả nhóm	Chưa bắt đầu
21	Điều phối nhóm: họp tuần, theo dõi tiến độ, review code, ghép nhánh	Ghép nối các khối giữa ba thành viên (Hình 2.1, 2.2)	1–5	5	TV1	Đang làm

3.2. Tiến độ theo tuần



Hình 3.1. Biểu đồ Gantt kế hoạch 5 tuần

Các hạng mục được xếp để mỗi người làm từ 4 đến 15 giờ mỗi tuần, chưa tính dự phòng; tuần 5 chỉ còn đo chi phí, báo cáo và demo. Bốn mốc kiểm soát, mỗi mốc lưu một bản chạy được:

- **M1, cuối tuần 2:** luồng chính chạy trọn vẹn từ nộp job đến tải kết quả.
- **M2, cuối tuần 3:** tự co giãn, 2 gói và xử lý Spot bị thu hồi hoạt động.
- **M3, cuối tuần 4:** đủ tính năng, dừng thêm tính năng mới.
- **M4, tuần 5:** nộp báo cáo và demo.

TÀI LIỆU THAM KHẢO

1. P. Mell, T. Grance. The NIST Definition of Cloud Computing, NIST SP 800-145, 2011.
2. AWS. Best practices for Amazon EC2 Spot.
3. AWS. Spot Instance interruption notices.
4. AWS. Scaling policy based on Amazon SQS.
5. AWS. Amazon SQS standard queues.
6. DoiT. AWS Public IPv4 Price Increase: The Complete Guide.
7. AWS. Amazon EC2 Spot Instance Advisor.
8. AWS. Track your Free Tier usage for Amazon EC2.